

TALENT INTELLIGENCE PLATFORM

200 Technical Interview Questions & Spoken Candidate Answers

This guide compiles 200 comprehensive interview questions covering the complete Talent Intelligence Platform backend microservices stack. The answers are structured exactly as a candidate would speak during a professional interview at service-based and product-based software companies. Topics covered include Microservices, Service Discovery, API Gateway, Auth Sync, Event-Driven Architecture (Kafka), S3 integration, Databases, Redis ZSets, PgVector Store, RAG Pipeline Retrieval, LLM Tools Integration, and scaling strategy.

PREPARED FOR TECHNICAL INTERVIEW PREPARATION
STACK: JAVA 21, SPRING BOOT 3.4.0, KAFKA, REDIS, PGVECTOR & OLLAMA

Section 1: Microservices, Eureka, and Routing Architecture (Q1 - Q50)

Q1: What is the high-level architecture of your project, and why did you choose it?

Candidate Answer: "Well, our system is built as an AI-powered recruitment platform using a Java and Spring Boot microservices architecture. We have 10 microservices, including a Service Gateway, Service Registry, Authentication, User Management, Resume Management, AI Screening, Candidate Ranking, Chat, and Notification. We chose this microservices pattern because each service has a distinct responsibility. For instance, the AI screening service, which consumes CPU-heavy local LLM endpoints, is completely decoupled from resume CRUD or user profile management. This separation allows us to scale services independently and ensures that a slowdown in the AI pipeline doesn't bring down the main user dashboard."

Q2: How does Eureka Service Registry work in your system, and how do services locate each other?

Candidate Answer: "Yeah, so we use Spring Cloud Netflix Eureka Server as our discovery agent, which runs on port 8761. When any of our microservices start, they register themselves with Eureka using their name - like 'resume-management-service' - and their dynamic port. When the API Gateway or services like Candidate Ranking need to call other services, they don't hardcode IP addresses. Instead, they lookup the registered service names in Eureka, and Spring Cloud LoadBalancer dynamically routes the HTTP requests using Round Robin."

Q3: How does the API Gateway route traffic to different microservices?

Candidate Answer: "Actually, the Spring Cloud Gateway acts as the single entry point on port 8090. We configured route predicates in its application.yml. When a request matches a pattern - for instance, /api/v1/auth/** or /api/v1/resumes/** - the gateway forwards it to the lb://authentication-service or lb://resume-management-service based on the Eureka registry details. This hides all internal port mappings from the client."

Q4: How is authentication handled in this microservices setup? Do you validate JWT in every service?

Candidate Answer: "No, validating JWT in every single service would be a huge performance bottleneck. Instead, we implemented a Gateway-level security pattern. When a request hits the API Gateway on port 8090, a custom AuthenticationFilter intercepts it, extracts the Bearer token from the Authorization header, validates the signature, and decodes the claims. If valid, the gateway mutates the request headers to inject 'X-User-Email', 'X-User-Role', and 'X-User-Id' before forwarding it to internal services. Internal services run a simple GatewayHeaderAuthFilter that trusts these headers and sets up Spring Security context instantly."

Q5: What happens if a user signs up? Explain the OTP registration flow.

Candidate Answer: "Well, we designed a multi-step signup process. When a user requests a signup at `/signup-otp`, their registration metadata is cached in a `PendingUserCacheService`, and an OTP event is published to Kafka. The Notification Service reads this event and sends an email. Once the user submits the correct OTP at `/verify-otp`, the user is officially saved in the `auth_users` table, and an authentication event is emitted to Kafka to sync their recruiter profile in user management. This ensures that only verified email addresses can access the system."

Q6: How did you solve the recruiter profile synchronization between Authentication and User Management services?

Candidate Answer: "Yeah, that was critical because we wanted to keep User Management decoupled from Auth. When a user successfully verifies their OTP, the Authentication service publishes an event to the `auth-user-registered` Kafka topic. The User Management service runs an `AuthEventConsumer` that listens to this topic, extracts the email, full name, and company, and saves it to the `recruiters` table. If the User Management service is down, Kafka stores the message, and it gets processed as soon as the service comes back online, achieving eventual consistency."

Q7: What databases are you using, and what is the database strategy?

Candidate Answer: "We use a Database-per-Service pattern. Every microservice has its own PostgreSQL database schema running on containerized PostgreSQL on port 5435. For example, Authentication uses `auth_db`, User Management uses `user_db`, Resume Management uses `resume_db`, and AI Screening uses `screening_db` equipped with pgvector. This prevents tight database-level coupling and allows us to customize database configurations independently."

Q8: How do your internal services communicate with each other?

Candidate Answer: "We use both synchronous and asynchronous communication. For read-heavy operations where we need immediate data, we use Spring Cloud OpenFeign to make REST calls. For example, Candidate Ranking queries Job Description and Resume Management synchronously using Feign Clients. However, for write-heavy, long-running processes - like resume processing or sync operations - we use Kafka messages to achieve asynchronous decoupling."

Q9: How do you handle CORS issues in your API Gateway?

Candidate Answer: "Well, we ran into CORS issues initially since the frontend runs on port 5173 and the gateway runs on 8090. To fix this, we created a custom `CorsConfig` class in our API Gateway. We configured a `CorsWebFilter` that allows requests from our React origin, permits standard methods like GET, POST, PUT, DELETE, and OPTIONS, and registers headers like Authorization, Content-Type, and user context headers."

Q10: How is the Admin recruiter user created in the system?

Candidate Answer: "Actually, we wanted to ensure the platform has at least one system admin profile at startup. So, in the Authentication service, we implemented an `AdminSeeder` class that implements Spring's `CommandLineRunner`. During startup, it checks if an admin user with email `admin@talent.com` exists in the database. If not, it hashes the password using BCrypt and writes the user with an ADMIN role type to the database."

Q11: What is the role of the GatewayHeaderAuthFilter class in downstream services?

Candidate Answer: "Yeah, so downstream services like Resume Management or Ranking are behind the API Gateway, but they still need to know who the logged-in user is. The `GatewayHeaderAuthFilter` is a standard servlet filter that extends `OncePerRequestFilter`. It reads `X-User-Email` and `X-User-Role` headers injected by the API Gateway. If present, it creates a `UsernamePasswordAuthenticationToken` and sets it in Spring's `SecurityContextHolder`, allowing standard Spring Security annotations like `@PreAuthorize` to work."

Q12: What hashing algorithm do you use to store passwords in your Authentication service?

Candidate Answer: "We use BCrypt hashing. Spring Security's `PasswordEncoder` is configured as a `BCryptPasswordEncoder` bean. When a user registers or resets their password, we hash it using BCrypt before saving it to the `password_hash` column. During login, the authentication provider uses BCrypt's internal verification to match the incoming plain text password against the hashed database value."

Q13: How did you manage user registration tokens? Do you use cookies or headers?

Candidate Answer: "We use JWT (JSON Web Tokens) sent in HTTP headers. Once a user logs in, the Auth service generates a signed JWT containing claims like email, userId, roles, and company name. This token is returned in the API response. The frontend React client stores this token in state/localStorage and sends it in the `Authorization: Bearer <token>` header for subsequent requests."

Q14: What is the token expiration strategy in your JWT service?

Candidate Answer: "Well, we configured the JWT to expire in 1 hour. This provides a balance between user convenience and security. The expiration claim (`exp`) is set dynamically when generating the claims. If a token is older than 1 hour, the API Gateway's JWT parser throws an `ExpiredJwtException`, and the request is rejected with a 401 Unauthorized status, prompting the frontend to log the user out."

Q15: How does the PendingUserCacheService store users before OTP verification?

Candidate Answer: "Actually, when a user starts a signup, we don't write their details directly to the ``auth_users`` table because they aren't verified yet. Instead, we store their credentials in a thread-safe map inside ``PendingUserCacheService`` (or Redis in production) keyed by their email. We also start an OTP countdown. Once they submit the correct OTP within the time limit, we fetch their details from this cache, write them to PostgreSQL, and clear the cache."

Q16: How does the OTP service generate and validate the verification codes?

Candidate Answer: "Yeah, so the ``OtpService`` uses a secure random generator to create a 6-digit numeric string. This code is stored in Redis (or a local cache) with a TTL of 5 minutes, mapped against the user's email. When the user verifies, we retrieve the cached OTP. If it matches the user's input, validation succeeds and we delete the OTP from the cache immediately to prevent reuse."

Q17: What roles are configured in your system, and how are they mapped to users?

Candidate Answer: "We have a ``RoleType`` enum containing roles like ``RECRUITER`` and ``ADMIN``. In the database, we use an element collection table ``auth_user_roles`` linked to the ``auth_users`` table. This allows a user to have multiple roles. The roles are injected as a list of strings into the JWT claims, which the API gateway parses and maps to roles in Spring Security."

Q18: How is the Eureka Server configured to handle failover or self-preservation?

Candidate Answer: "Well, in development, we disabled client register-with-eureka and fetch-registry on the Eureka server itself using YAML settings. In production, we keep self-preservation enabled. This prevents Eureka from immediately evicting services during transient network glitches when heartbeats are missed, ensuring the registry remains stable."

Q19: How does the User Management service handle updates to recruiter profiles?

Candidate Answer: "Actually, ``user-management-service`` exposes endpoints in ``UserController`` to update profile fields like full name, company name, and phone. We use JPA's ``@PreUpdate`` lifecycle hook to automatically set the ``updated_at`` timestamp. This allows us to track when a recruiter's contact information was last modified."

Q20: What happens if a Feign client call fails? Do you have fallback mechanisms?

Candidate Answer: "Yes, we use Resilience4j with OpenFeign to handle failures. If a service call fails - for example, if Candidate Ranking tries to fetch details from Resume Management and times out - it falls back to a default method or logs a warning and returns a placeholder object, ensuring the ranking process doesn't throw a 500 error."

Q21: How do you enforce role-based access control (RBAC) in your microservices?

Candidate Answer: "We use Spring Security's `@PreAuthorize` annotation on our controllers. Since the `GatewayHeaderAuthFilter` parses the `X-User-Role` header and assigns granted authorities like `ROLE_RECRUITER` or `ROLE_ADMIN`, we can secure endpoints using expressions like `@PreAuthorize("hasRole('ADMIN')")` to restrict actions."

Q22: What security dependencies did you import into your pom.xml for authentication?

Candidate Answer: "We imported `spring-boot-starter-security` for Spring Security integration, and the `jjwt-api`, `jjwt-impl`, and `jjwt-jackson` libraries from `io.jsonwebtoken` to handle JWT generation, serialization, and signature verification."

Q23: How do you handle password reset requests in your Authentication service?

Candidate Answer: "Yeah, so the user calls `/forgot-password` with their email. We verify if they exist, generate a reset OTP, and publish it to Kafka. The user then submits this OTP along with their new password to `/reset-password`. We validate the OTP, hash the new password using BCrypt, update the database record, and delete the OTP cache."

Q24: How does your API Gateway locate the Eureka Server?

Candidate Answer: "Well, we configured the gateway's `eureka.client.serviceUrl.defaultZone` property to point to `http://localhost:8761/eureka/`. During bootstrap, the gateway registers itself and queries Eureka to cache service instances locally for routing."

Q25: What is the difference between registration-service and user-profile service in your system?

Candidate Answer: "Actually, the `authentication-service` is responsible only for credentials, security, and credentials check (JWT issues). The `user-management-service` is responsible for business metrics - like recruiter profiles, company listings, and active status. This keeps security concerns separate from business logic."

Q26: How does your Gateway load balancer decide which instance to route to if there are multiple service instances?

Candidate Answer: "Spring Cloud Gateway uses Reactor LoadBalancer under the hood. It queries Eureka for a list of active instances for a service name and distributes requests among them using Round Robin. This prevents overloading any single instance."

Q27: How is JWT signing key generated in your security configuration?

Candidate Answer: "We use a secret key string defined in our properties file. We convert it to bytes and pass it to `Keys.hmacShaKeyFor()` from the JJWT library. This generates a secure HMAC-SHA signing key used to sign and parse the JWT tokens."

Q28: Why did you use Java 21 for this project?

Candidate Answer: "Well, Java 21 brings virtual threads (Project Loom) which are lightweight threads that drastically reduce resource consumption for I/O-bound microservices. It also offers pattern matching and record classes, making our DTOs cleaner and our asynchronous pipelines much more performant under load."

Q29: How do you handle validation errors in DTOs, like invalid emails or blank fields?

Candidate Answer: "We use Jakarta validation annotations like `@NotBlank`, `@Email`, and `@Size` in our request DTOs. If a validation fails, Spring throws a `MethodArgumentNotValidException`. We intercept this in a `@RestControllerAdvice` class and return a structured 400 Bad Request response with details of each validation error."

Q30: What is the default zones setup in eureka client configurations?

Candidate Answer: "It's the endpoint address of the Eureka server, typically `http://localhost:8761/eureka/`. Every microservice client has this defined under `eureka.client.service-url.defaultZone` so it knows where to register and query for service registries."

Q31: How does your Gateway filter handle pre-flight CORS requests (OPTIONS method)?

Candidate Answer: "Actually, pre-flight requests do not carry JWT tokens. So, in our `AuthenticationFilter`, we check if the incoming request is an `OPTIONS` method. If it is, we bypass JWT check, set the HTTP response status to 200 OK, and immediately return. This prevents pre-flight requests from being rejected as unauthorized."

Q32: What is a record class, and why did you use it for DTOs in some services?

Candidate Answer: "A Java record is an immutable data class. It automatically generates getters, `toString`, `equals`, and `hashCode` at compile time. We used records for DTOs because they represent raw data structures, reducing boilerplate code significantly."

Q33: Why does your system use a separate Database for Authentication and User Management?

Candidate Answer: "If they shared a database, a database failure would take down both auth and profile access. By keeping them separate, we can update schemas independently, scale databases differently, and secure Auth credentials database (`auth_db`) with stricter firewalls than profile metadata."

Q34: How do you trace client requests across multiple microservices?

Candidate Answer: "Well, we use Micrometer Tracing (formerly Spring Cloud Sleuth) with Zipkin. When a request hits the API Gateway, it generates a unique trace ID. This ID is propagated through headers in Feign client calls and Kafka messages. This allows us to trace a request across all services in the logs."

Q35: What is the database initialization strategy for your project?

Candidate Answer: "We use Hibernate's auto-ddl update feature in development (`spring.jpa.hibernate.ddl-auto: update`). This automatically creates and updates tables when the microservices start, saving us from writing manual migrations during active development."

Q36: How do you handle exceptions in your microservices? Do you return raw stack traces to users?

Candidate Answer: "No, returning raw stack traces is a security risk and looks unprofessional. We use a `@ControllerAdvice` global exception handler in each service. It catches custom exceptions (like `UserNotFoundException`) and system errors, wrapping them in a clean JSON format containing status, error message, and timestamp."

Q37: How does the notification service authenticate with Gmail SMTP?

Candidate Answer: "We configured Spring Mail settings in the notification service `application.yml`. We specify `smtp.gmail.com` as host, port `587`, and use Gmail App Passwords rather than actual passwords to authenticate. We also enable STARTTLS to encrypt the connection."

Q38: What happens if the Authentication service database is down when a user signs up?

Candidate Answer: "If the database is down, the service cannot save the user. In this case, the `/signup-otp` endpoint throws a connection exception. Our global exception handler intercepts it, logs the trace, and returns a 503 Service Unavailable response to the user."

Q39: How did you configure port allocation to avoid local conflicts?

Candidate Answer: "Yeah, we allocated ports systematically. Eureka Server runs on 8761, Gateway on 8090, Auth on 8091, User Management on 8069, Resume on 8083, JD on 8084, Screening on 8085, Ranking on 8086, Chat on 8087, and Notification on 8088. This avoids conflicts during local runs."

Q40: How does your Gateway mutate headers? Why is request mutation necessary?

Candidate Answer: "Actually, HTTP requests are immutable by default in Spring WebFlux. To add custom headers, we must call the `mutate()` method on the request object. This allows the API Gateway to inject decoded claims (like `X-User-Email`) and pass the mutated request down the filter chain."

Q41: What happens if a JWT has an invalid signature?

Candidate Answer: "The signature validation check in JJWT fails and throws a `SignatureException`. The API Gateway filter catches this exception, prints a warning log, halts the request chain, and returns an HTTP 401 Unauthorized response to the client."

Q42: How do you prevent SQL Injection in your database operations?

Candidate Answer: "We use Spring Data JPA (Hibernate) which automatically generates parameterized queries under the hood. Parameterized inputs prevent user input from modifying SQL statement structures, thereby mitigating SQL Injection risks completely."

Q43: What is the purpose of Spring Cloud OpenFeign?

Candidate Answer: "OpenFeign is a declarative web service client. Instead of writing boilerplate code using `RestTemplate` or `WebClient`, we define a Java interface, annotate it with `@FeignClient`, and write method signatures. Spring automatically generates the implementation to execute REST calls."

Q44: How do you secure your backend endpoints if a user bypasses the API Gateway and calls them directly?

Candidate Answer: "Downstream services require authentication. Since they use `GatewayHeaderAuthFilter`, they will block any request that lacks the `X-User-Email` and `X-User-Role` headers. We also run them inside a private virtual network where only the API Gateway is exposed to the public internet."

Q45: Why is the `@PrePersist` annotation used in entities?

Candidate Answer: "We use `@PrePersist` to execute a callback function before Hibernate performs the insert query. It is very useful for automatically setting fields like `created_at` or `updated_at` without manual intervention."

Q46: How does your system handle horizontal scaling? What happens if you run 3 instances of Authentication service?

Candidate Answer: "Since they register with Eureka, the API Gateway load balancer will distribute requests across all 3 instances. We store session state in a distributed cache like Redis (or JWT itself), making our Authentication service completely stateless and scalable."

Q47: What is self-preservation mode in Eureka Server?

Candidate Answer: "Self-preservation is a feature where Eureka registry stops evicting expired service instances if it detects a sudden drop in heartbeats. This protects the system from network partitions where services are active but cannot heartbeat Eureka."

Q48: What is the role of Lombok in your Java code?

Candidate Answer: "Lombok reduces boilerplates in Java by generating constructors, getters, setters, builders, and log objects at compile time using simple annotations like `@Getter`, `@Setter`, and `@Builder`."

Q49: Why is BCrypt considered safe for password hashing?

Candidate Answer: "BCrypt incorporates a salt to protect against rainbow table attacks and is a slow, computationally expensive algorithm. We can adjust the work factor (cost) to make password cracking extremely slow even with high computational power."

Q50: How do you verify if all your microservices are healthy and running?

Candidate Answer: "We integrated Spring Boot Actuator in all services. By calling `/actuator/health`, we get health indicators. We can also view Eureka dashboard at `http://localhost:8761` to check the online registration status of each microservice."

Section 2: Resume Storage, DB Transactions, and State Tracking (Q51 -

Q51: What is the flow of the resume upload in the platform? Explain from frontend to database.

Candidate Answer: "Well, in our platform, the recruiter selects a PDF or DOCX file on the React frontend. The client issues a multipart POST request containing the file, recruiter ID, and candidate details to the API Gateway at `http://localhost:8090/api/v1/resumes/upload`. The gateway routes this to `resume-management-service`. The service first generates a unique key, uploads the file binary to an AWS S3 bucket via our `StorageService`, and inserts a record in our `candidate_resumes` table with a status of `UPLOADED`. It then publishes a `resume-uploaded` event to Kafka and returns a success response with the new resume ID to the frontend."

Q52: How does S3 storage integration work? What AWS credentials do you use?

Candidate Answer: "Actually, we use Spring Cloud AWS or standard AWS SDK to build our `AwsS3Config` bean. We configure the access key, secret key, region, and bucket name in our `application.yml`. The `StorageService` takes the `MultipartFile`, extracts its input stream, creates an `ObjectMetadata` with the correct content length, and uses the `AmazonS3` client's `putObject` method to store it. The file URL returned follows the standard S3 virtual host addressing."

Q53: Why do we track different states like UPLOADED, PARSED, SCREENED, and FAILED?

Candidate Answer: "Yeah, because resume parsing and AI screening are long, asynchronous operations that take several seconds. Tracking these states allows us to provide a reactive user interface. When a resume is first uploaded, the UI shows a 'Processing...' indicator. As soon as the background worker chunks it, the state changes to `PARSED`, and once Ollama finishes grading it, it updates to `SCREENED`. If anything fails (like a corrupt file or network timeout), it moves to `FAILED` so the recruiter can see the exact failure."

Q54: What is the column definition of rawText and fileUrl columns in candidate_resumes and job_descriptions tables?

Candidate Answer: "Well, for `file_url`, we set the SQL length limit to `512` characters since S3 URLs can be quite long. For `raw_text` in the `job_descriptions` table, we configured `columnDefinition = 'TEXT'` because job descriptions can contain thousands of words which exceed the default VARCHAR limit of 255. This maps to the PostgreSQL `TEXT` data type, which has no practical length limit."

Q55: What is the purpose of `@ElementCollection` in `JobDescription` entity?

Candidate Answer: "Actually, we wanted to store a list of key skills (like Java, Kafka, Spring) directly linked to a job description without creating a separate full-fledged Entity class. `@ElementCollection` allows us to create a collection of basic types (strings) that Hibernate maps to a secondary join table named `job_skills`. This keeps our database design clean and avoids the overhead of managing a complex `@OneToMany` relationship."

Q56: How do you fetch a candidate's resume binary for rendering in the browser?

Candidate Answer: "In `ResumeController`, we expose a GET endpoint that accepts the resume ID. We fetch the file metadata from PostgreSQL to get the S3 URL. Then, the `StorageService` calls the S3 client to download the object as an `S3ObjectInputStream`. We wrap this stream in a Spring `InputStreamResource` and return it in an `ResponseEntity` along with the correct `Content-Type` header (like `application/pdf`) and a `Content-Disposition` header set to inline. This allows the browser to render the PDF directly in a viewer instead of downloading it."

Q57: What is the database configuration of `resume-management-service`? Which database does it connect to?

Candidate Answer: "It connects to `resume_db` running on PostgreSQL on port 5435. In its `application.yml`, the `spring.datasource.url` is set to `jdbc:postgresql://localhost:5435/resume_db` with username and password. We also configure connection pool properties like Hikari CP maximum pool size to ensure performance under load."

Q58: How does the `CandidateResume` entity mapping handle candidate experience?

Candidate Answer: "Yeah, the entity has a `totalExperience` field typed as `Double`. We chose `Double` instead of an integer because candidate experience can often be fractional - for instance, 1.5 years or 4.5 years. This field is initially populated as null (or 0.0) during upload, and then updated with the precise value extracted from the resume text by the AI screening pipeline."

Q59: How do you handle transactional safety when uploading a resume and writing to the database?

Candidate Answer: "Well, we use `@Transactional` annotations on our service methods. However, since S3 uploads are external network operations, we do not wrap the S3 upload inside a database transaction block. If S3 upload fails, we throw an exception and do not write to the database. If S3 succeeds but database write fails, we delete the uploaded file from S3 in a catch block to prevent orphaned files in our cloud bucket."

Q60: How do you implement pagination when retrieving a list of resumes for a recruiter?

Candidate Answer: "We use Spring Data JPA's `PagingAndSortingRepository`. In our repository interface, we define a method like `findByRecruiterId(UUID recruiterId, Pageable pageable)`. The controller accepts page and size parameters, maps them to a `PageRequest.of(page, size)`, and passes it to the service. This returns a `Page<CandidateResume>` object containing the list along with total pages and elements, making it easy to build pagination controls on our React UI."

Q61: What is the purpose of `@PreUpdate` in your `CandidateResume` entity?

Candidate Answer: "It allows us to write a callback method that executes automatically before an update statement is run on the database. In our entity, we have a method annotated with `@PreUpdate` that sets the `updatedAt` field to `LocalDateTime.now()`. This guarantees that our record timestamps are always accurate without us having to write setter calls in our service layer."

Q62: How do you configure Hibernate's `ddl-auto` property in your microservices?

Candidate Answer: "In our `application.yml` profiles, we configure `spring.jpa.hibernate.ddl-auto` to `update` for local development. This ensures tables are created automatically on startup. For production or staging, we set it to `validate` and use Liquibase to execute migration scripts, ensuring that schema alterations are audited and safe."

Q63: What happens if a resume is archived by a recruiter?

Candidate Answer: "Actually, we implement soft deletes. The `CandidateResume` entity has a boolean field `archived` defaulting to false. When a recruiter deletes a resume, we update this flag to true instead of running a SQL `DELETE`. When listing active resumes, our repository filter queries only where `archived = false`. This keeps candidate histories for audits while hiding them from active dashboards."

Q64: Why do you use UUIDs instead of auto-incrementing Long IDs for entities like `CandidateResume`?

Candidate Answer: "Well, using sequential Long IDs in a distributed system is risky. If an attacker guesses a resume ID (like 1, 2, 3), they can try to scrape data. Additionally, UUIDs can be generated on the client or in services without checking the database, which prevents ID conflicts when merging databases or syncing distributed records."

Q65: How do you handle bulk uploads of resumes? Does the API Gateway choke?

Candidate Answer: "No, because the API Gateway acts as a non-blocking reactive gateway using Netty. When a bulk upload is triggered, the frontend sends concurrent requests. Since the gateway is asynchronous, it forwards them efficiently. The `resume-management-service` handles the uploads and immediately offloads processing to Kafka, so the service doesn't block while waiting for parsing."

Q66: How does the `KafkaEventPublisher` class dispatch events in `resume-management-service`?

Candidate Answer: "Yeah, so the `KafkaEventPublisher` wraps Spring's `KafkaTemplate`. It takes an event payload - like a map containing the resume ID and file URL - and calls `kafkaTemplate.send()`. We also add callback listeners to the future object returned by Spring to log whether the message was successfully dispatched to the broker or if it failed, which helps us debug Kafka connection issues."

Q67: How does the `ResumeStatusConsumer` process status updates from other services?

Candidate Answer: "It is annotated with `@KafkaListener(topics = \"resume-status-updated\")`. When it receives a message containing the resume ID and status (like `PARSED` or `SCREENED`), it parses the JSON map, calls the repository to find the resume by its UUID, updates the status field, and saves it. This keeps the state aligned across services."

Q68: What is the JSON configuration for Kafka message serialization in your resume service?

Candidate Answer: "We configure our `KafkaTemplate` to use `JsonSerializer` for the value. In the application settings, we configure `spring.kafka.producer.value-serializer` as `org.springframework.kafka.support.serializer.JsonSerializer`. This automatically converts Java maps or DTOs into clean JSON strings before publishing to topics."

Q69: How do you handle transient connection issues with S3? Do you retry?

Candidate Answer: "Yes, in our S3 client configuration, we configure a custom `ClientConfiguration` object. We set properties like connection timeout, client execution timeout, and max error retries to `3`. If a request fails due to transient network congestion, the SDK automatically retries before throwing an exception."

Q70: What happens if a resume metadata update request is sent via Feign to an endpoint that doesn't exist?

Candidate Answer: "If the endpoint is incorrect or the signature mismatches, Feign throws a `FeignException\$NotFound` or `FeignException\$Unauthorized`. We catch this in our service layer or use an error decoder to unpack the exact HTTP status and handle the error gracefully."

Q71: How does the frontend check the status of a resume after uploading?

Candidate Answer: "Actually, the React client runs a polling mechanism or uses WebSockets. After upload, it checks the endpoint `/api/v1/resumes/{id}` every 3 seconds to get the latest status. Once it reads the status as `PARSED` or `SCREENED`, it updates the state, stops polling, and renders the parsed metadata."

Q72: How do you map Enum types in JPA? Why did you use String instead of Ordinal for RoleType?

Candidate Answer: "We use the `@Enumerated(EnumType.STRING)` annotation. If we used `@Enumerated(EnumType.ORDINAL)`, Hibernate would save the enum indexes (0, 1, 2) in the database. If we later added a new enum value at the beginning, all database mappings would break. Storing them as strings (like 'RECRUITER') is safer and makes database audits much easier."

Q73: What is the use of `@Column(nullable = false)` in Hibernate entities?

Candidate Answer: "It adds a NOT NULL constraint to the database column when Hibernate generates schema. It also acts as a primary check: if our service layer tries to persist an entity with a null value in that field, JPA throws a validation exception before executing the SQL statement, keeping the database consistent."

Q74: How do you prevent LazyInitializationException in Spring Boot JPA?

Candidate Answer: "We configure fetch types carefully. For example, in our `JobDescription` entity, the `@ElementCollection` of `keySkills` is configured with `fetch = FetchType.EAGER` to ensure skills are loaded in the same query. For lazy associations, we execute JPQL queries with `JOIN FETCH` inside our repositories to pull nested data in a single transactional query."

Q75: How do you verify files uploaded are actual PDFs and not malicious executables?

Candidate Answer: "Well, we validate the file on two layers. First, on the frontend, we specify `accept='.pdf,.docx'` on the input element. Second, in `ResumeController`, we check the content type of the `MultipartFile` and check the file extension. If it's not a PDF or DOCX, we reject the upload with a 400 Bad Request before calling S3."

Q76: What is the purpose of `@JsonIgnore` in entities with relationships?

Candidate Answer: "It prevents infinite recursion during JSON serialization. If Entity A has a relationship with Entity B, and B links back to A, Jackson will loop infinitely trying to serialize them. `@JsonIgnore` breaks this loop by telling Jackson not to serialize the reverse link."

Q77: How do you configure JPA database dialect in your configuration?

Candidate Answer: "In our `application.yml`, we define `spring.jpa.properties.hibernate.dialect: org.hibernate.dialect.PostgreSQLDialect`. This tells Hibernate to generate SQL syntax optimized specifically for PostgreSQL features like UUID generation and pgvector columns."

Q78: What is HikariCP, and why is it included in Spring Boot?

Candidate Answer: "HikariCP is a high-performance database connection pool. Spring Boot imports it by default. It manages a pool of active database connections, so our microservices don't have to spend overhead opening and closing TCP connections for every single HTTP request, speeding up query times."

Q79: How do you fetch all resumes uploaded by a specific recruiter?

Candidate Answer: "We expose a GET endpoint `/api/v1/resumes/recruiter/{recruiterId}` in `ResumeController`. It validates the recruiter exists and queries `resumeRepository.findByRecruiterId(recruiterId)`. It returns a list of candidate resumes, which the React app uses to display the candidate dashboard."

Q80: What happens if two users upload resumes at the exact same millisecond? How does the database handle this concurrency?

Candidate Answer: "Since they are generated with random UUIDs and handled as separate HTTP requests in Spring's thread pool, Hibernate inserts them as distinct records in parallel. The database handles the locking and writes, so they do not collide."

Q81: How does the `StorageService` download files from S3? Do you download to disk?

Candidate Answer: "No, downloading to disk would fill up the microservice's storage and slow down execution. Instead, we stream the file. The `StorageService` fetches the S3 object, reads the stream directly, and pipes it straight to the HTTP response output stream, which is memory-efficient."

Q82: How did you set up database credentials securely in docker-compose.yml?

Candidate Answer: "In our `docker-compose.yml` file, we defined environment variables like `POSTGRES_USER` and `POSTGRES_PASSWORD` with values. For production, we inject these values from host environment variables or secret vault services rather than hardcoding them in files."

Q83: How do you handle database index creation for the resumes table?

Candidate Answer: "We configure indexes using JPA annotations. In our entity definitions, we define indexes under `@Table` options. For instance, we index the `recruiter_id` and `candidate_email` columns because we query them frequently, reducing lookup times from linear scans to logarithmic searches."

Q84: What is the difference between `@GeneratedValue(strategy = GenerationType.AUTO)` and `IDENTITY` in Hibernate?

Candidate Answer: "Actually, `IDENTITY` relies on database auto-increment columns (like serial in PG), whereas `AUTO` lets Hibernate select the strategy based on the database driver. For UUID fields, we use `GenerationType.AUTO` which tells Hibernate to generate UUID values using its internal random generator."

Q85: How do you structure your DTO classes in Java? What is their purpose?

Candidate Answer: "DTOs (Data Transfer Objects) isolate our internal JPA database entities from the API layer. We create request and response DTOs (e.g., `CreateJobRequest`) containing only the fields required for that API call. This prevents exposure of sensitive entity fields and allows us to change database models without breaking external API contracts."

Q86: How does Spring Boot resolve active profiles like dev and prod?

Candidate Answer: "Spring Boot reads the `spring.profiles.active` property from command-line arguments, system variables, or the default `application.properties`. If we run `java -jar app.jar --spring.profiles.active=prod`, it loads the `application-prod.yml` configuration over the base settings."

Q87: What is the role of the `@Repository` annotation in Spring Boot?

Candidate Answer: "It registers the class as a Spring-managed Bean and enables automatic exception translation, converting low-level database driver exceptions into Spring's unified `DataAccessException` hierarchy."

Q88: Why is it important to use a connection pool size limit?

Candidate Answer: "If we don't limit connection pool size, a spike in API traffic could spawn thousands of database connections, exhausting database memory and CPU. Setting a limit (like 10-20 connections per service instance) queues requests gracefully and keeps the database healthy."

Q89: What is the advantage of using `@Builder` annotation from Lombok?

Candidate Answer: "It implements the Builder design pattern, allowing us to instantiate complex entities with multiple fields in a readable way, like `CandidateResume.builder().id(uuid).status("UPLOADED").build()`, without writing telescopic constructors."

Q90: How do you configure spring datasource driver class name?

Candidate Answer: "By default, Spring Boot auto-detects the driver class from the connection URL prefix. However, we can specify it explicitly in properties as `spring.datasource.driver-class-name: org.postgresql.Driver` to ensure the correct PostgreSQL driver is loaded."

Q91: What is the purpose of `@Column(updatable = false)` on `created\_at` field?

Candidate Answer: "It tells Hibernate to exclude this column from any SQL UPDATE queries it generates. This ensures that once the creation timestamp is written during insert, it can never be altered by subsequent updates, maintaining record integrity."

Q92: How do you handle cross-microservice transactions? Do you use 2-Phase Commit (2PC)?

Candidate Answer: "No, 2-Phase Commit is slow and blocks databases in microservices. Instead, we use the Saga pattern with eventual consistency. When an event occurs, the source service writes to its DB and publishes a Kafka event. Downstream services listen to this event and update their databases. If a failure occurs, compensating events are dispatched to undo previous states."

Q93: How does `resume-management-service` know when the AI screening has finished grading a resume?

Candidate Answer: "Yeah, so when the AI screening is complete, `ai-screening-service` makes a Feign client call to `resume-management-service`'s endpoint `/internal/resumes/{id}/status` to update the state to `SCREENED`. If the Feign call fails, it falls back to publishing a Kafka event to the status topic, which the resume service consumes."

Q94: What is the use of Hibernate's `@PrePersist` annotation?

Candidate Answer: "It designates a method to run right before an entity is inserted into the database. We use it to set default values, such as setting `archived = false` and initializing timestamps, ensuring consistent state values."

Q95: What does `@NoArgsConstructor` do, and why is it required by Hibernate?

Candidate Answer: "It generates a constructor with no arguments. Hibernate requires a default no-arg constructor to instantiate entities when fetching records from the database using reflection before mapping column values to fields."

Q96: How do you ensure that S3 file URLs are secure and cannot be accessed publicly by anyone?

Candidate Answer: "We configure our S3 bucket as private, blocking all public read access. When a user requests a file, they call our resume service. Our service authenticates the request and generates a temporary pre-signed S3 URL with a 5-minute expiration time, ensuring files are accessed only by authorized users."

Q97: What is Spring Data JPA, and how does it differ from raw Hibernate?

Candidate Answer: "Hibernate is the actual ORM engine that maps database tables to Java objects. Spring Data JPA is an abstraction layer on top of Hibernate. It provides repository interfaces (like `JpaRepository`) that generate queries dynamically based on method names, reducing SQL boilerplate code."

Q98: How do you configure the Spring Boot application name in your services?

Candidate Answer: "We specify it in the `application.yml` under the property `spring.application.name`. This name (e.g. `resume-management-service`) is used by the Eureka registry to identify the service and by other services to reference it via Feign clients."

Q99: What is the `@Entity` annotation?

Candidate Answer: "It is a Jakarta Persistence annotation that specifies that the class is a database Entity, mapping it directly to a database table."

Section 3: Event-Driven Kafka Pipelines & System Fault Tolerance (Q100-Q103)

Q100: Why did you use Apache Kafka in this project instead of simple REST APIs?

Candidate Answer: "Well, we chose Kafka because resume parsing, text embedding, and AI screening are heavy, time-consuming tasks. If we used synchronous REST APIs, the frontend would have to keep the HTTP connection open for 15-20 seconds while Ollama processes the files. This would block thread pools and lead to timeouts. With Kafka, the resume service saves the file to S3, writes a quick database record, publishes a ``resume-uploaded`` event, and immediately returns a 201 response. The AI Screening service consumes this event asynchronously in the background. If the screening service goes offline, Kafka queues the events, ensuring no uploads are lost."

Q101: How does ``ResumeUploadedConsumer`` listen to Kafka topics in ``ai-screening-service``?

Candidate Answer: "Actually, we created a consumer class annotated with ``@Service``. We define a method annotated with ``@KafkaListener(topics = "resume-uploaded", groupId = "screening-group")``. When a message is published to the ``resume-uploaded`` topic, Kafka delivers it to this listener method. The method accepts the record payload as a Map, extracts the resume ID and file URL, and invokes the resume indexing pipeline."

Q102: What configuration did you write in `application.yml` for Kafka serializer and deserializer?

Candidate Answer: "Yeah, so for the producer, we configure ``key-serializer`` as ``StringSerializer`` and ``value-serializer`` as ``JsonSerializer``. For the consumer, we configure ``key-deserializer`` as ``StringDeserializer`` and the ``value-deserializer`` as ``ErrorHandlingDeserializer``. The value delegate deserializer is set to ``JsonDeserializer`` so it converts JSON strings back into Java Map objects dynamically."

Q103: Why is ``ErrorHandlingDeserializer`` used in your consumer configuration?

Candidate Answer: "Well, that was a crucial design decision. If a producer publishes a message that the consumer cannot deserialize (for instance, due to a class mismatch or a corrupt JSON string), a standard ``JsonDeserializer`` will throw an exception and loop indefinitely trying to re-read the bad message, clogging the partition. ``ErrorHandlingDeserializer`` intercepts these deserialization errors, wraps the bad record, and forwards it to an error handler or DLQ instead of blocking the consumer thread."

Q104: What was Bug B/E in your Kafka event publisher, and how did you resolve it?

Candidate Answer: "Ah, that was a tricky issue! Initially, `publishStatusUpdate` published a custom Java class `ResumeStatusUpdatedEvent` to the Kafka topic. Jackson added a `__TypeId__` header containing the full class package name to the message metadata. When the consumer in `resume-management-service` tried to read it, Jackson looked for that exact Java class package. Since the classes were in different packages across services, it threw a `MismatchedInputException` and routed messages to the Dead Letter Queue. We fixed this by publishing messages as plain `HashMap` objects. This removed the class headers and allowed safe, cross-service deserialization."

Q105: How do you handle Dead Letter Queues (DLQ) in your Kafka setup?

Candidate Answer: "Actually, we configure a `CommonErrorHandler` bean in our Kafka configuration. If a consumer throws an exception while processing a message (like a database connection timeout), Spring Kafka tries to process the message up to 3 times with a short backoff. If it still fails, the error handler automatically sends the bad message to a Dead Letter Topic named `<original-topic>.DLT`. This allows us to inspect failed events manually without interrupting the active processing stream."

Q106: How did you coordinate the async transitions from UPLOADED to PARSED, and then to SCREENED?

Candidate Answer: "Yeah, so it's a step-by-step event pipeline. First, the upload endpoint writes `UPLOADED` state and sends `resume-uploaded` to Kafka. Next, `ai-screening-service` consumes this, extracts text, writes vectors to PGVector, publishes `resume-parsed` event, and calls Feign to update status to `PARSED`. Finally, `candidate-ranking-service` consumes the `resume-parsed` event, performs job screening, generates scores, and publishes a status change event to update the resume to `SCREENED`."

Q107: What happens if a Kafka consumer is offline when a message is published?

Candidate Answer: "Kafka retains messages in partitions on disk for a configured retention period (default: 7 days). When the offline consumer comes back online with the same consumer group ID, it checks its last saved offset in Kafka, retrieves all unread messages from that offset onward, and processes them sequentially. This guarantees that no messages are lost during service downtimes."

Q108: What is the purpose of consumer groups in Apache Kafka?

Candidate Answer: "Consumer groups allow us to scale message consumption horizontally. If we run 3 instances of our `ai-screening-service` under the same group ID `screening-group`, Kafka distributes the topic partitions among these 3 instances. Each partition is consumed by only one instance in the group, allowing us to load balance message processing without double-processing data."

Q109: How do you define the key for your Kafka messages? Why is it important?

Candidate Answer: "We use the ``resumeld`` as the message key. Kafka uses the key's hash value to determine which partition the message is routed to. By using ``resumeld`` as the key, we guarantee that all events related to a specific resume (upload, parse, status updates) are routed to the exact same partition. This ensures that the messages are processed in the strict chronological order in which they were published."

Q110: How does ``notification-service`` interact with Kafka?

Candidate Answer: "Well, ``notification-service`` runs a ``NotificationConsumer`` that listens to topics like ``auth-user-registered`` or ``notification-request``. When it consumes an event containing user details and notification context, it invokes its ``EmailService`` to dispatch an email. This decouples email generation delays from our core services."

Q111: What is the role of Zookeeper in your Docker Compose Kafka configuration?

Candidate Answer: "In our setup, Kafka uses Zookeeper to manage cluster metadata, coordinate broker states, choose partition leaders, and perform node membership tracking. Zookeeper acts as the coordination engine for our Kafka broker container."

Q112: How do you prevent duplicate message processing in your consumers (Idempotency)?

Candidate Answer: "Actually, we enforce idempotency at the database layer. In our consumers, before processing an event, we check if the target state has already been reached. For example, if we consume a ``resume-status-updated`` event to mark status as ``SCREENED``, we first query the resume record. If the status is already ``SCREENED``, we log a skip message and return immediately. This prevents duplicate writes."

Q113: What is the default retention period in Kafka, and how do you configure it?

Candidate Answer: "By default, Kafka retains messages for 168 hours (7 days). We can override this in our docker-compose config or server.properties using the ``log.retention.hours`` setting. For our local environment, we reduced this to 24 hours to save local disk space."

Q114: Why is the ``spring.json.trusted.packages`` property set to ``*`` in consumer configs?

Candidate Answer: "Yeah, Spring's ``JsonDeserializer`` restricts classes it will deserialize for security reasons. Setting ``spring.json.trusted.packages: "*"``` tells the deserializer to trust all packages, which is necessary since we pass untyped maps and dynamic structures across separate microservices."

Q115: What happens if a Kafka consumer encounters a database timeout? Does it block other messages?

Candidate Answer: "If it encounters a timeout, it throws an exception. Our configured error handler executes retries. During the retry phase, the consumer thread is blocked from fetching subsequent messages in that partition. This is normal because it preserves ordering. Once retries are exhausted and the message goes to the DLQ, processing resumes."

Q116: How do you configure the bootstrap server location in Spring Boot?

Candidate Answer: "We define it in `application.yml` under `spring.kafka.bootstrap-servers` with value `localhost:9092`. In containerized environments, we change this to `kafka:29092` so the services can find the broker within the Docker network."

Q117: What is a bootstrap server in Kafka?

Candidate Answer: "A bootstrap server is an initial Kafka broker address that the client connects to during startup. The client queries this broker to fetch the complete cluster metadata, including lists of other brokers, topics, and partitions."

Q118: How does `ai-screening-service` handle resume deletions in Kafka?

Candidate Answer: "Well, when a recruiter deletes a resume in the resume service, it publishes a `resume-deleted` event to Kafka. The `ai-screening-service` runs a `ResumeDeletedConsumer` that listens to this topic, extracts the resume UUID, and invokes `deleteVectorsAndChunks` to remove all text chunks and vector embeddings from PostgreSQL."

Q119: How does the candidate-ranking service consumer update scores when a resume is parsed?

Candidate Answer: "The `candidate-ranking-service` runs a `ResumeParsedConsumer` listening to `resume-parsed` topic. When a message arrives, it gets the recruiter ID, fetches all job descriptions for that recruiter from `job-description-service` via Feign, and calls `RankingService.calculateAndStoreScore` for each job to rank the new candidate."

Q120: What is the difference between a partition and a topic in Kafka?

Candidate Answer: "A topic is a logical category where messages are sent, like 'resume-uploaded'. A partition is a physical log file on the Kafka broker where messages of a topic are stored. Topics are split into multiple partitions for scalability and parallel processing."

Q121: How do you scale Kafka consumers to handle high traffic spikes?

Candidate Answer: "We can increase the number of partitions for our topics. If a topic has 6 partitions, we can scale our consumer service up to 6 instances in the same consumer group. Kafka will assign one partition to each instance, enabling parallel execution."

Q122: How does Spring Boot's `@KafkaListener` handle JSON conversion?

Candidate Answer: "Spring uses a `RecordMessageConverter` bean. If a consumer method parameter is typed as a Map or Java object, Spring's messaging converter automatically uses Jackson to deserialize the incoming JSON byte payload into that object."

Q123: What configuration prevents class structure conflicts in Kafka events?

Candidate Answer: "We set `spring.json.use.type.headers` to `false` in our producer configuration. This tells Kafka not to write Java class metadata headers into the message, allowing downstream services to deserialize the JSON independently."

Q124: What is an offset in Kafka?

Candidate Answer: "An offset is a unique sequential integer assigned to each message within a partition. It acts as a cursor: the broker uses offsets to track which messages have been read by which consumer groups."

Q125: How do you manually commit offsets in a consumer?

Candidate Answer: "We configure `spring.kafka.listener.ack-mode` to `MANUAL`. In our listener method, we add an `Acknowledgment` parameter and call `ack.acknowledge()` after the database operation succeeds. This guarantees at-least-once delivery."

Q126: What happens if a Kafka consumer takes too long to process a message?

Candidate Answer: "If processing takes longer than `max.poll.interval.ms`, Kafka assumes the consumer is dead, kicks it out of the group, and triggers a partition rebalance. We configure this interval to be large enough to accommodate AI processing times."

Q127: How does the query router event flow work in recruiter chat?

Candidate Answer: "Actually, once the chat assistant completes a query response, it publishes a `chat.interaction.completed` event to Kafka with details like conversation ID and message content. This allows audit logs to process chat histories asynchronously."

Q128: Why is Kafka highly performant compared to activeMQ?

Candidate Answer: "Kafka uses disk storage with sequential log append operations and zero-copy transfer mechanisms, letting it handle millions of messages per second. ActiveMQ manages complex queue states in memory, which limits throughput."

Q129: How does your system handle backpressure in Kafka?

Candidate Answer: "We configure the `max.poll.records` property in our consumer to limit the number of messages fetched in a single poll. This prevents the worker thread pool from being overwhelmed with too many tasks at once."

Q130: How do you ensure message delivery guarantees (e.g. at-least-once)?

Candidate Answer: "We configure the producer `acks` setting to `all` to ensure all partition replicas acknowledge the write. In the consumer, we commit offsets only after database operations succeed, ensuring no message is dropped."

Q131: What is partition rebalancing in Kafka?

Candidate Answer: "Rebalancing is the process where Kafka redistributes partition assignments among consumers in a group, triggered when a consumer node joins or leaves the group."

Q132: How did you set up Zookeeper client connection ports in docker?

Candidate Answer: "In our `docker-compose.yml`, we mapped Zookeeper's internal client port 2181, which the Kafka container uses to communicate with Zookeeper on startup."

Q133: Why does `candidate-ranking-service` consume both `resume-parsed` and `resume-deleted` events?

Candidate Answer: "It consumes `resume-parsed` to score new candidates, and `resume-deleted` to remove old screening reports from PostgreSQL and clear scores from Redis ZSets, maintaining data synchronization."

Q134: What is the default group ID setting in Spring Boot Kafka consumers?

Candidate Answer: "It is configured via the property `spring.kafka.consumer.group-id`. If not specified globally, we must set it on the `@KafkaListener(groupId = ...)` annotation to distinguish consumer groups."

Q135: How do you configure connection timeouts for Kafka brokers?

Candidate Answer: "We use the setting `spring.kafka.properties.request.timeout.ms` in our configurations. This forces the client to abort a request if the broker takes too long to respond."

Q136: What is a message deserializer?

Candidate Answer: "It is a utility that converts raw byte arrays received from Kafka brokers back into structured Java objects or strings."

Q137: What happens if a Kafka broker goes down?

Candidate Answer: "If we have a multi-broker setup, Kafka automatically promotes a sync replica partition to leader, allowing clients to failover and continue operations without data loss."

Q138: Why is class coupling a problem in microservices messaging?

Candidate Answer: "If services share compiled Java classes for messages, a change in one service forces us to update and deploy the other. Using plain JSON structures avoids this coupling."

Q139: How do you verify your Kafka docker container is running?

Candidate Answer: "We can run ``docker ps`` to verify the container state, or check container logs using ``docker logs talent-kafka`` to verify broker initialization."

Q140: What is a replica in Kafka?

Candidate Answer: "A replica is a redundant copy of a partition log stored on another broker node to guarantee high availability and fault tolerance."

Q141: How does ``user-management-service`` process auth sync events safely?

Candidate Answer: "It validates that the incoming payload contains all required fields, writes profile records inside a ``@Transactional`` service call, and catches exceptions to prevent consumer failures."

Q142: What is the role of ``JsonDeserializer`` trusted packages configuration?

Candidate Answer: "It restricts which Java packages are allowed to be deserialized, protecting the application from malicious classpath object execution attempts."

Q143: How do you configure the consumer auto offset reset property?

Candidate Answer: "We set ``spring.kafka.consumer.auto-offset-reset`` to ``earliest``. This forces a new consumer group to read the topic from the very first message rather than only reading new messages."

Q144: What happens if a consumer throws a runtime exception in Spring Boot?

Candidate Answer: "Spring's listener container intercepts the exception, logs it, and delegates it to the configured error handler to trigger retries or DLQ routing."

Q145: Why do you use a serialization delegate class in value deserializer?

Candidate Answer: "It acts as a safety wrapper. If serialization fails, the delegate catches the exception and returns a failed object container, allowing the main consumer thread to inspect the error."

Q146: What are Kafka headers?

Candidate Answer: "Headers are key-value metadata pairs attached to a Kafka record, separate from the main payload, used for tracing, routing, and type mapping."

Q147: How do you configure local Kafka ports in docker compose?

Candidate Answer: "We map port `9092:9092` for host connections (like local microservices), and define an internal listener on `29092` for container-to-container communication."

Q148: How does Kafka guarantee message order?

Candidate Answer: "Kafka guarantees strict message ordering only within a single partition. As long as messages share the same key, they write to the same partition and process in order."

Q149: What is a dead letter topic?

Candidate Answer: "A dead letter topic is a designated topic where messages that fail repeatedly during processing are routed for debugging, preventing pipeline blockages."

Section 4: Advanced RAG, AI Copilot, pgvector, and Business Strategy (

Q150: What is the RAG pipeline architecture in your recruiter chat service?

Candidate Answer: "Well, we built a highly sophisticated Retrieval-Augmented Generation (RAG) system in our `recruiter-chat-service`. When a recruiter asks a question about a candidate, the pipeline first routes the query using a `QueryRouterService` to decide if it requires vector retrieval, a tool execution, or is conversational. If it requires retrieval, we execute a multi-query expansion and query rewriting. Then, we execute a hybrid search, combining keyword and vector similarity search against the candidate's chunks. The retrieved chunks are filtered and reordered by our `RerankingService`, and the top context chunks are appended to the system prompt, which is sent to Ollama along with chat history from `RedisChatMemory` to generate the answer."

Q151: How does `pgvector` store and query embeddings in PostgreSQL?

Candidate Answer: "Actually, `pgvector` is a PostgreSQL extension that adds support for vector data types. In our database, Spring AI's `PgVectorStore` creates a table called `vector\_store` with an `embedding` column of type `vector(768)` (matching the dimension of the `nomic-embed-text` model). When we query, it converts the search prompt to a vector and uses the cosine distance operator (`<=>`) inside a SQL query - like `SELECT \* FROM vector\_store ORDER BY embedding <=> ? LIMIT ?` - to retrieve the closest semantic chunks."

Q152: What is hybrid retrieval, and why did you implement it?

Candidate Answer: "Yeah, so standard vector search matches concepts but can miss specific keywords like version numbers or product names. Keyword search matches exact words but misses semantic context. Our `HybridDocumentRetriever` runs both queries: a vector similarity search on pgvector, and a keyword search on our plaintext database. We then merge and deduplicate the results, ensuring our chat assistant doesn't miss specific technical details."

Q153: How does the `RerankingService` optimize the retrieved document chunks?

Candidate Answer: "Well, the retrieval step can fetch a lot of noisy or irrelevant chunks. Sending all of them to the LLM increases latency and can confuse the model. Our `RerankingService` calculates a secondary relevance score for each chunk relative to the user query using a cross-encoder model (or a fast similarity metric). It filters out low-scoring chunks and reorders the survivors so the most relevant information sits at the top of the LLM's context window."

Q154: How does the chatbot persist conversation history across requests?

Candidate Answer: "We use `RedisChatMemory` to store conversation histories. In Spring AI, we configure a `MessageChatMemoryAdvisor` and pass a conversation ID. Under the hood, this advisor retrieves the last N turns of the chat history from Redis, format-maps them to message structures, and appends them to the LLM prompt. New messages are automatically appended to Redis, making the chatbot stateful and thread-safe."

Q155: Explain Bug G/H: Llama3 template rendering conflict, and how you fixed it.

Candidate Answer: "Ah! In Spring AI, the default `ChatClient` uses `.param("name", value)` to bind variables to system prompts. However, Ollama's local `llama3` parser has its own internal templating system. When Spring AI sent parameters in headers, llama3 threw a 'template string is not valid' parsing exception, failing the pipeline. We resolved this by bypassing Spring AI's parameter binding and doing plain string formatting and concatenation in Java before calling the prompt, resolving the template conflicts."

Q156: How does `recruiter-chat-service` execute LLM tools? How does Universal Chat work?

Candidate Answer: "Actually, if a recruiter is in Universal Chat mode (no single candidate selected), the system prompt instructs the LLM that it can query records. We register our `RecruiterAssistantTools` class containing beans like `listAllCandidates` or `candidateRanking` with the `ChatClient`. When the user asks a question like 'Who is the top candidate for Job X?', the LLM calls the tool dynamically. Spring executes the underlying Java method, fetches database records, and returns them as context for the LLM to write the response."

Q157: What embedding model are you using, and what is its dimension size?

Candidate Answer: "We use `omicron-embed-text` running locally on Ollama. It generates embeddings with a dimension of `768`. We configured this dimension size in our `application.yml` using `spring.ai.vectorstore.pgvector.dimension: 768` to ensure the database vector column aligns with the model's output."

Q158: What is the fallback heuristic if the AI screening service is offline?

Candidate Answer: "Well, in `RankingService`, if calling the AI screening service fails, we catch the exception and fall back to a vector-embedding heuristic. We compute the cosine similarity between the job description vector and each resume chunk vector using the `EmbeddingModel` interface. We average these similarities to calculate a match score and perform keyword-overlap matching for skills with a threshold of 0.55, ensuring ranking still works."

Q159: What are the tool options available in `RecruiterAssistantTools`?

Candidate Answer: "We have `listAllCandidates` (returns candidate resume listing), `candidateLookup` (searches candidates by name/skills), `listAllJobs` (lists available vacancies), `candidateRanking` (returns ranked match reports for a job), and `interviewQuestion` (generates structured interview questions for a candidate)."

Q160: How does Apache Tika extract text from PDF and DOCX files in your system?

Candidate Answer: "We created a `TikaParser` class. It downloads the S3 file as an input stream, creates an Apache Tika `Parser` instance, a `BodyContentHandler`, and a `Metadata` object. When it calls `parser.parse()`, Tika automatically detects the document format and extracts all text content into a single string stream."

Q161: What is the chunking strategy you implemented? Why is it necessary?

Candidate Answer: "We implemented a sliding window character-based chunker in `TextChunker`. We use a chunk size of `1000` characters with an overlap of `200` characters. Chunking is necessary because LLMs have token context limits. By breaking the resume into smaller overlapping pieces, we can fetch only the highly relevant chunks during RAG searches, saving memory and keeping context precise."

Q162: How does the `QueryRouterService` classify user queries?

Candidate Answer: "It uses a lightweight prompt classification call to Ollama. It evaluates the user's input and classifies it into one of three routes: `VECTOR\_SEARCH` (if querying candidate skills or specific history), `TOOL\_CALL` (if asking to list candidates, jobs, or run rankings), or `CONVERSATIONAL` (for general greetings or follow-ups)."

Q163: How do you secure Redis cache keys to prevent collision?

Candidate Answer: "In our `RedisChatMemory` configuration, we prefix all session keys with a unique namespace, like `chat:session:`. This guarantees that chat histories do not collide with other Redis cache keys used for ranking or user sessions."

Q164: What is the cosine similarity calculation, and how did you implement it in Java?

Candidate Answer: "Cosine similarity measures the cosine of the angle between two multi-dimensional vectors. We implemented it by calculating the dot product of the two float arrays and dividing it by the product of their Euclidean norms, returning a value between -1.0 and 1.0 representing semantic closeness."

Q165: How does Spring AI support Feign clients and service discovery?

Candidate Answer: "Actually, Spring AI doesn't integrate with Feign directly, but our microservices do. We use Spring Cloud OpenFeign to query other Spring services. The fetched data is wrapped in Spring AI `Document` metadata before indexing or feeding to LLM pipelines."

Q166: How does local LLM execution affect latency compared to cloud APIs?

Candidate Answer: "Well, local Ollama execution is completely free and private, but latency is dependent on the host machine's hardware (GPU/CPU). Cloud APIs (like OpenAI or Gemini) are faster but incur usage costs and send private candidate resume files over the network. For development, local Ollama works great; in production, we swap it for high-performance cloud LLMs."

Q167: Why is the `initialize-schema` property set to `true` in PgVector settings?

Candidate Answer: "It tells Spring AI to execute database script checks on startup. If the `vector\_store` table or pgvector extension is not present in PostgreSQL, Spring AI automatically initializes them, simplifying setup."

Q168: How do you handle PDF files that are scanned images instead of text? Does Tika parse them?

Candidate Answer: "Standard Tika extracts embedded text. If a PDF is a scanned image, standard extraction returns empty. To handle this, we can integrate Tika with OCR (Tesseract) on the host machine. If empty, the pipeline logs a warning and marks the record state as `FAILED` or falls back to basic metadata."

Q169: What is the difference between a Cross-Encoder and a Bi-Encoder in RAG?

Candidate Answer: "A Bi-Encoder encodes the query and document separately to calculate cosine similarity (very fast, used for retrieval). A Cross-Encoder processes the query and document together to calculate interaction (highly accurate but slower, used for Reranking)."

Q170: What is a product-based company perspective on building this system?

Candidate Answer: "A product-based perspective focuses on scalability, high availability, stateless architecture, and tenancy. We design the system to handle millions of users, optimize CPU-intensive Ollama calls, build multi-tenant databases to keep customer data isolated, and focus on lowering runtime costs."

Q171: What is a service-based company perspective on building this system?

Candidate Answer: "A service-based perspective focuses on rapid feature delivery, meeting client requirements, ease of maintenance, clean code documentation, and integration flexibility. The architecture must be easy to customize if the client wants to swap Ollama for OpenAI or swap PostgreSQL for MongoDB."

Q172: How would you optimize the system to handle 10,000 resume uploads per hour?

Candidate Answer: "Well, we would horizontally scale the `resume-management` and `ai-screening` services by running multiple instances. We would scale Kafka partitions for topics, scale S3 throughput, use GPU-accelerated Ollama instances, and implement caching for embeddings to prevent redundant LLM calculations."

Q173: How do you handle data privacy in this recruitment application?

Candidate Answer: "Data privacy is critical since resumes contain PII. We encrypt files at rest in S3, use private VPCs for databases, and run Ollama locally. Since Ollama runs on-premise, candidate data never leaves our network, satisfying GDPR and HIPAA requirements."

Q174: How would you handle a scenario where the AI screening service takes 60 seconds to process a resume?

Candidate Answer: "Actually, because the pipeline is asynchronous via Kafka, a 60-second processing time does not affect the REST API performance. The message stays in the partition, and the worker processes it in the background. The user sees a 'Processing...' state on the React UI, which updates reactively once complete."

Q175: What index type would you create on the vector_store table in pgvector for production?

Candidate Answer: "For production with large datasets, we would create an HNSW (Hierarchical Navigable Small World) index or an IVFFlat index. HNSW provides fast, high-recall search times for vector similarity queries, though it takes more memory."

Q176: How do you manage local dependencies when running the microservices?

Candidate Answer: "We use a multi-module Maven parent POM that controls all version numbers, and a docker-compose file that spins up PostgreSQL, Redis, and Kafka in containers. We also wrote a PowerShell start script (start.ps1) to compile and launch all services systematically."

Q177: How would you implement multi-tenancy in the resume management database?

Candidate Answer: "We would use a shared-database, discriminator-column approach by adding a tenant_id column to all tables. Every SQL query would filter by tenant_id = current_tenant. For stronger isolation, we could use a database-per-tenant schema setup."

Q178: What is the difference between keyword retrieval and semantic retrieval?

Candidate Answer: "Keyword retrieval matches exact terms (like 'Java 17') using text indices. Semantic retrieval matches the meaning of sentences (like 'modern programming expertise') using vector distance, even if they don't share exact words."

Q179: How do you handle memory leaks in long-running Java microservices?

Candidate Answer: "We analyze Heap dumps using Eclipse Memory Analyzer, configure JVM heap limits (-Xmx), use clean resource management (try-with-resources for streams), and avoid storing large datasets in static maps, letting the Garbage Collector do its job."

Q180: How does candidate ranking order search results?

Candidate Answer: "We query the `screening\_reports` table, sorting records descending by the `match\_score` column. At runtime, we map ranks sequentially (1st, 2nd, 3rd) and return the list. We also mirror this scoring order inside our Redis ZSets."

Q181: What is the purpose of Spring AI's VectorStore interface?

Candidate Answer: "It provides a unified abstraction over multiple vector databases (PGVector, Pinecone, Milvus, Chroma). This allows us to switch our vector database by changing configuration properties without editing our core RAG code."

Q182: How does `api-gateway` route requests to Eureka registered services?

Candidate Answer: "It uses the `lb://` prefix inside the route URI configurations. Spring Cloud Gateway's load balancer interceptor translates this into actual instances registered under that service name in Eureka."

Q183: What is the role of Docker volumes in your docker-compose.yml?

Candidate Answer: "We use a volume mapping `pgdata:/var/lib/postgresql/data` for our PostgreSQL container. This ensures that when container instances are restarted or deleted, our database files remain persistent on the host disk."

Q184: Why is the `@PreAuthorize` annotation useful?

Candidate Answer: "It allows us to secure API endpoints declaratively using expression-based syntax, verifying user roles before executing the target controller method, preventing unauthorized API calls."

Q185: How would you handle a memory leak in the Apache Tika parser?

Candidate Answer: "We would wrap Tika parser calls inside a separate JVM process or use an external Tika server container. If it experiences a memory leak while parsing a large file, only that container restarts without affecting our AI Screening service."

Q186: How do you write Unit tests for your service classes in Spring Boot?

Candidate Answer: "We use JUnit 5 and Mockito. We annotate our test classes with `@ExtendWith(MockitoExtension.class)`, mock external dependencies (like repositories or Feign clients) using `@Mock`, and verify business logic behaviors using Assertions."

Q187: What is the difference between `@RestController` and `@Controller`?

Candidate Answer: "`@RestController` is a convenience annotation that combines `@Controller` and `@ResponseBody`. It tells Spring to write the returned objects directly to the HTTP response body as JSON instead of rendering an HTML view template."

Q188: How would you handle high database load in your User Management service?

Candidate Answer: "We would implement caching using Spring Cache abstraction with Redis. We would cache recruiter profiles by their ID. Subsequent read calls would fetch from Redis, reducing the database query load."

Q189: How do you configure the thread pool size for Spring Boot's asynchronous tasks?

Candidate Answer: "We can define a custom `TaskExecutor` bean in a configuration class and configure parameters like core pool size, max pool size, and queue capacity to control resource utilization."

Q190: Why did you use docker-compose instead of standard Kubernetes for development?

Candidate Answer: "Docker Compose is lightweight, fast to spin up, and runs locally with minimal configuration. It is perfect for local developer environments, while Kubernetes is designed for staging and production clustering."

Q191: What is Project Loom virtual threads, and how do they benefit Spring Boot microservices?

Candidate Answer: "Virtual threads are lightweight threads managed by the JVM rather than the OS. They allow us to handle millions of concurrent blocking operations (like database queries or HTTP calls) with minimal memory overhead, boosting throughput."

Q192: How does `recruiter-chat-service` map active job details into prompt context?

Candidate Answer: "If a `jobId` is present in the request context, the service queries `job-description-service` via Feign client. It appends the raw text of the job description to the system prompt, providing the AI with role details."

Q193: What is the advantage of using a multi-module Maven project setup?

Candidate Answer: "It allows us to manage all microservices as a single cohesive project. We declare all dependency versions in the parent POM, ensuring version consistency and enabling single-command builds for the entire platform."

Q194: How do you secure connection details like database passwords in production configurations?

Candidate Answer: "We use placeholder properties in our configurations, like `#{DB_PASSWORD}`, and inject the actual sensitive values from secure environment variables or vault systems at runtime, keeping passwords out of source code repositories."

Q195: What is the purpose of `@Qualifier` in Spring injection?

Candidate Answer: "If a service has multiple beans implementing the same interface (like `ChatClient`), Spring will throw an exception during dependency injection. `@Qualifier` specifies the exact bean name (e.g. `screeningChatClient`) to inject."

Q196: How would you scale the Redis container if it becomes a bottleneck?

Candidate Answer: "We would run a Redis Cluster with multiple master and replica nodes, enable read-scaling on replicas, and use Redis Sentinel to manage failovers and high-availability routing."

Q197: How does the system ensure that a deleted resume cannot be searched anymore in the chat?

Candidate Answer: "When a delete request occurs, the deletion consumer deletes vectors from pgvector and chunks from the database immediately. The chat retrievers will find no records matching that ID, removing it from queries."

Q198: What makes this project a strong candidate portfolio piece?

Candidate Answer: "It covers a complete microservices architecture, integrates enterprise event-driven patterns with Kafka, uses a state-of-the-art AI stack with Spring AI and local LLMs (Ollama), implements advanced RAG search systems, and runs on a secure, transactional PostgreSQL and Redis setup."